Wprowadzenie do Swifta

Zaliczenie

- Ocena końcowa: średnia z laboratoriów i egzaminu
- Obie części trzeba zaliczyć na ocenę pozytywną
- Wykład: egzamin na Moodle na ostatnich zajęciach, pytania otwarte
- Laboratoria: sprawozdania w PDF na Moodle

Laboratoria

- Po każdym laboratorium sprawozdanie w PDF na Moodle
- Ze sprawozdań co najmniej ocena dostateczna

filip.wandzio@wroclaw.merito.pl

Plan wykładu

- Czym jest Swift
- Środowisko pracy: Xcode, toolchain, SPM
- Zmienne, stałe i typy danych
- Typ opcjonalny
- Warunki, pętle i kolekcje
- Funkcje i struktury
- Testy jednostkowe i integracyjne
- Laboratorium 1

Czym jest Swift?

- Język ogólnego przeznaczenia od Apple, pokazany w 2014 roku
- Następca Objective-C
- Kompilowany, kompilator oparty na LLVM
- Open source: macOS, Linux, Windows
- Aktualnie Swift 6.4

Gdzie używa się Swifta?

- Aplikacje Apple: iOS, iPadOS, macOS, watchOS, tvOS
- Narzędzia wiersza poleceń i skrypty
- Aplikacje serwerowe na Linuksie
- Zalety: czytelna składnia, bezpieczeństwo, wydajność

Xcode

- Darmowe IDE od Apple, z App Store, tylko macOS
- Debugger, symulatory, profiler, edytory UI, dokumentacja
- Nowa wersja raz w roku, razem z nowym iOS
- Publikacja w App Store: konto deweloperskie za 100 USD rocznie
- Na laboratoriach nie jest wymagany

Swift bez Xcode

- Toolchain: kompilator, debugger, biblioteka standardowa, menedżer pakietów
- Edytor dowolny

```
xcode-select --install      # macOS
swiftly install latest      # Linux, Windows
swift --version             # sprawdzenie instalacji

docker run --rm -it -v "$PWD":/pkg -w /pkg swift:6.4 bash
```

Swift Package Manager

- Nowy projekt: swift package init
- Na co dzień: build, run, test
- Pojedynczy plik: swift plik.swift

```
mkdir kalkulator && cd kalkulator
swift package init --type executable

swift build     # kompilacja
swift run       # kompilacja i uruchomienie
swift test      # testy
```

Manifest Package.swift

- Opis projektu, też w Swifcie
- Pierwsza linia to wersja narzędzi, nie komentarz

```
// swift-tools-version:6.0
import PackageDescription

let package = Package(
    name: "kalkulator",
    targets: [
        .executableTarget(name: "kalkulator")
    ]
)
```

Składnia w pigułce

```swift
let jezyk = "Swift"
var rok = 2014
rok = 2026
print("\(jezyk) ma już \(rok - 2014) lat")
```

Biblioteka standardowa a Foundation

- Biblioteka standardowa: zawsze dostępna
- print, readLine, Int, Double, String, Array, Dictionary, min, max, abs
- Foundation: wymaga import Foundation
- String(format:), Date, sqrt, pow, FileManager

```
let bok = pole.squareRoot()  // zamiast sqrt(pole) z Foundation
```

Hello, World!

```
print("Hello, World!")
print("Podaj swoje imię")
let name = readLine()
print("Hello, \(name)!")
```

- Wynik: Hello, Optional("Adam")!
- readLine zwraca String?, bo wejścia może zabraknąć

Rozpakowanie wartości z readLine()

- ?? podaje wartość domyślną
- Konwersja tekstu na liczbę też może się nie udać

```
let name = readLine() ?? "świecie"
print("Hello, \(name)!")

let age = Int(readLine() ?? "") ?? 0
print("Za rok będziesz mieć \(age + 1) lat")
```

Komentarze

```
// komentarz liniowy

/* komentarz
   blokowy */

// MARK: sekcja
// TODO: do zrobienia
```

Formatowanie kodu

- swift format jest w toolchainie
- Reguły w pliku .swift-format
- Formatter uruchamiamy przed wklejeniem kodu do sprawozdania

```
swift format --in-place -r Sources Tests        # popraw
swift format lint --strict -r Sources Tests     # tylko zgłoś
```

Zmienne i stałe

```
var licznik = 100
licznik = 101

let pi = 3.14159
let cena: Double = 100
```

Typy danych

```
let liczba: Int = -522
let pi: Float = 3.14
let precyzja: Double = 3.14159
let zdane: Bool = true
let litera: Character = "S"
let tekst: String = "Hello"
```

Operatory arytmetyczne

```
let dzielna = 17
let dzielnik = 5

print(dzielna / dzielnik)  // 3
print(dzielna % dzielnik)  // 2
print(dzielna + 2.5)       // błąd: Int + Double
```

Konwersja typów

```
let dystans = 30
let czas = 0.25
let predkosc = Double(dystans) / czas  // 120.0

let liczba = Int("42")                 // Int?
```

Łączenie tekstu

```
let cm = 10 * 2.54

print("Wynik: " + String(cm) + " cm")
print("Wynik: \(cm) cm")
print("10 cali to \(10 * 2.54) cm")
```

Oficjalny przykład: proste wartości

```swift
var myVariable = 42
myVariable = 50
let myConstant = 42

let explicitDouble: Double = 70

let label = "The width is "
let width = 94
let widthLabel = label + String(width)

let apples = 3
let oranges = 5
let fruitSummary = "I have \(apples + oranges) pieces of fruit."
```

- Źródło: [1], A Swift Tour, Simple Values

Typ opcjonalny

- T? to wartość typu T albo nil
- T? nie da się użyć tam, gdzie potrzebne jest T
- ! na nil kończy program błędem

```
var nickName: String?                    // String albo nil

let phone = "601123321"
let phoneNumber: Int? = Int(phone)

let number = phoneNumber ?? 0            // bezpiecznie: wartość domyślna
let forced = phoneNumber!                // wymuszone rozpakowanie
```

Oficjalny przykład: typy opcjonalne

- == nil sprawdza, czy wartości brakuje
- ?? podstawia wartość domyślną

```
var optionalString: String? = "Hello"
print(optionalString == nil)
// Prints "false".

let nickname: String? = nil
let fullName: String = "John Appleseed"
let informalGreeting = "Hi \(nickname ?? fullName)"
```

Przykład: cale na centymetry

```
print("Podaj cale: ")
let cale = Double(readLine() ?? "") ?? 0
print("Jest to \(cale * 2.54) cm.")
```

- readLine() zwraca String?, ?? "" daje pusty tekst
- Double(...) zwraca Double?, ?? 0, gdy tekst nie jest liczbą
- Interpolacja wstawia wynik bez ręcznej konwersji

if i else

- Warunek bez nawiasów, klamry obowiązkowe
- Warunek musi być typu Bool, liczba nie wystarczy

```
let temperatura = 23

if temperatura > 25 {
    print("gorąco")
} else if temperatura > 15 {
    print("ciepło")
} else {
    print("zimno")
}

let opis = temperatura > 15 ? "ciepło" : "zimno"
```

switch

- Musi obsłużyć każdą wartość, stąd default
- Bez break i bez przechodzenia do następnego case
- Zakresy i kilka wartości w jednym case

```
let ocena = 4

switch ocena {
case 5:
    print("bardzo dobry")
case 3, 4:
    print("dobry albo dostateczny")
case 1..<3:
    print("do poprawy")
default:
    print("nie ma takiej oceny")
}
```

if let i guard let

- if let rozpakowuje opcjonal tylko wewnątrz klamer
- guard let rozpakowuje do końca funkcji, else musi z niej wyjść

```
let wiek = Int(readLine() ?? "")

if let wiek {
    print("Za rok: \(wiek + 1)")
} else {
    print("To nie jest liczba")
}

func powitanie(_ imie: String?) -> String {
    guard let imie else {
        return "Cześć!"
    }
    return "Cześć, \(imie)!"
}
```

Pętla for-in

- 1...5 to zakres domknięty, 0..<5 bez górnej granicy
- stride: krok inny niż 1

```
for numer in 1...5 {
    print(numer)                    // 1 2 3 4 5
}

for indeks in 0..<3 {
    print(indeks)                   // 0 1 2
}

for liczba in stride(from: 10, to: 0, by: -2) {
    print(liczba)                   // 10 8 6 4 2
}

for owoc in ["jabłko", "gruszka"] {
    print(owoc)
}
```

while i repeat-while

- repeat-while wykona ciało co najmniej raz
- break kończy pętlę, continue przechodzi do następnego obrotu

```
var licznik = 3
while licznik > 0 {
    print(licznik)
    licznik -= 1
}

var odpowiedz = ""
repeat {
    print("Kontynuować? (t/n)")
    odpowiedz = readLine() ?? "n"
} while odpowiedz != "t" && odpowiedz != "n"
```

Tablice i słowniki

```
var oceny = [5, 4, 3]
oceny.append(5)
print(oceny.count)          // 4
print(oceny[0])             // 5

var stolice = ["Polska": "Warszawa", "Czechy": "Praga"]
stolice["Niemcy"] = "Berlin"
print(stolice["Polska"] ?? "brak")

for (kraj, stolica) in stolice {
    print("\(kraj): \(stolica)")
}
```

- Słownik zwraca opcjonal, bo klucza może nie być

Funkcja: wywołanie

- Przy wywołaniu podajemy etykiety argumentów
- Zapis w dokumentacji: rectangleArea(width:height:)

```
let pole = rectangleArea(width: 3, height: 4)   // 12.0
let cm = centimeters(fromInches: 10)            // 25.4
```

Funkcja: definicja

- -> Double to typ wyniku, return zwraca wynik
- ofWeight weight: etykieta i nazwa w ciele funkcji

```
func rectangleArea(width: Double, height: Double) -> Double {
    return width * height
}

func bmi(ofWeight weight: Double, height: Double) -> Double {
    return weight / (height * height)
}

bmi(ofWeight: 80, height: 1.8)
```

Oficjalny przykład: funkcje

- _ to brak etykiety
- on to etykieta inna niż nazwa parametru

```
func greet(person: String, day: String) -> String {
    return "Hello \(person), today is \(day)."
}
greet(person: "Bob", day: "Tuesday")

func greet(_ person: String, on day: String) -> String {
    return "Hello \(person), today is \(day)."
}
greet("John", on: "Wednesday")
```

Struktura: kilka wartości naraz

- Equatable: porównanie operatorem ==

```
struct Rectangle: Equatable {
    let area: Double
    let perimeter: Double
}

func rectangle(width: Double, height: Double) -> Rectangle {
    return Rectangle(area: width * height, perimeter: 2 * (width + height))
}

let result = rectangle(width: 3, height: 4)
print(result.area)      // 12.0
print(result.perimeter)  // 14.0
```

Gdy wyniku nie ma: nil

- Typ wyniku Double?, gdy obliczenie może nie mieć sensu
- warunek ? a : b to operator trójargumentowy

```
func bmi(ofWeight weight: Double, height: Double) -> Double? {
    return height == 0 ? nil : weight / (height * height)
}

bmi(ofWeight: 80, height: 0)  // nil
bmi(ofWeight: 80, height: 2)  // Optional(20.0)
```

Stałe pogrupowane w enum

- enum bez przypadków jako przestrzeń nazw
- static let to stała typu
- Nazwana stała zamiast magicznej liczby

```
enum Geometry {
    static let degreesInFullAngle = 360
    static let pi = 3.141_592_653
}

let half = Geometry.degreesInFullAngle / 2  // 180
let obwod = 2 * Geometry.pi * 5.0
```

Swift Testing

- Framework testów w toolchainie od Swifta 6
- Nic nie trzeba instalować
- Uruchomienie: swift test

Projekt z testami

- Kod w Sources, testy w Tests
- swift package init od razu używa Swift Testing

```
mkdir lab1 && cd lab1
swift package init --type library
swift test

# Package.swift
# Sources/Lab1/Lab1.swift
# Tests/Lab1Tests/Lab1Tests.swift
```

Wygenerowany test

- import Testing: framework
- @testable import: dostęp do kodu z Sources
- @Test: funkcja jest testem

```
import Testing
@testable import Lab1

@Test func example() async throws {
    // Write your test here ...
}
```

Jak wygląda test

```
import Testing
@testable import Lab

@Suite struct RectangleTests {
    @Test func smallRectangle() {
        let result = rectangle(width: 3, height: 4)
        #expect(result.area == 12)
        #expect(result.perimeter == 14)
    }

    @Test("kwadrat 1 × 1")
    func unitSquare() {
        let expected = Rectangle(area: 1, perimeter: 4)
        #expect(rectangle(width: 1, height: 1) == expected)
    }
}
```

Oficjalny przykład: Swift Testing

```
import Testing

@Test func helloWorld() {
    let greeting = "Hello, world!"
    #expect(greeting == "Hello")
}
```

- Test celowo nie przechodzi
- Wynik: Expectation failed: (greeting → "Hello, world!") == "Hello"
- Komunikat pokazuje wartość faktyczną i oczekiwaną

#expect, #require i liczby Double

- #expect zapisuje porażkę, test biegnie dalej
- #require przerywa test, wymaga throws i try
- Double porównujemy z tolerancją, bo 0.1 + 0.2 != 0.3

```
@Test func bmiForTwoMeters() throws {
    #expect(bmi(ofWeight: 80, height: 0) == nil)

    let value = try #require(bmi(ofWeight: 80, height: 2))
    #expect(abs(value - 20) < 0.0001)
}
```

Uruchamianie i czytanie wyników

```
swift test                                          # wszystkie testy
swift test --filter RectangleTests                  # jeden zestaw
swift test --filter RectangleTests/unitSquare       # jeden test

# ✘ Test tenInches() recorded an issue at UnitConversionTests.swift:9:9:
#   Expectation failed: (centimeters(fromInches: 10) → 0.0) == 25.4
```

- Po strzałce: co zwróciła funkcja
- Po prawej: wartość oczekiwana

Testy jednostkowe i integracyjne

- Jednostkowy: jedna funkcja, np. arithmetic(of:and:)
- Integracyjny: cały program, od wejścia do wyjścia
- Każde laboratorium ma co najmniej jeden test integracyjny

Program jako funkcja

- Logika programu w funkcji: wejście to linie tekstu, wyjście to linie tekstu

```swift
// Sources/Lab1/Calculator.swift
public func runCalculator(input: [String]) -> [String] {
    let first = Double(input[0]) ?? 0
    let second = Double(input[1]) ?? 0
    let result = arithmetic(of: first, and: second)

    var output = ["Suma: \(result.sum)", "Iloczyn: \(result.product)"]
    guard let quotient = result.quotient else {
        output.append("Iloraz: dzielenie przez zero")
        return output
    }
    output.append("Iloraz: \(quotient)")
    return output
}
```

main.swift

- main.swift tylko czyta wejście i drukuje wynik

```swift
// Sources/Kalkulator/main.swift
var input: [String] = []
while let line = readLine() {
    input.append(line)
}
for line in runCalculator(input: input) {
    print(line)
}
```

Test integracyjny

```swift
@Test func sumOfTwoNumbers() {                          // jednostkowy
    #expect(arithmetic(of: 6, and: 3).sum == 9)
}

@Test func calculatorProgram() {                        // integracyjny
    let output = runCalculator(input: ["6", "0"])

    #expect(output == [
        "Suma: 6.0",
        "Iloczyn: 0.0",
        "Iloraz: dzielenie przez zero",
    ])
}
```

- Te same dane co z klawiatury, sprawdzamy dokładnie to, co zobaczy użytkownik

To samo z terminala

```
$ printf "6\n0\n" | swift run -q Kalkulator
Suma: 6.0
Iloczyn: 0.0
Iloraz: dzielenie przez zero
```

- Wynik programu musi zgadzać się z testem integracyjnym
- Ten przebieg wklejamy do sprawozdania jako przykład wejścia i wyjścia

Jak wyglądają laboratoria

- Treść laboratorium: PDF od prowadzącego
- Zadania u siebie, w dowolnym edytorze
- Rozwiązania: sprawozdanie w PDF

Sprawozdanie

- Strona tytułowa: przedmiot, temat, imię i nazwisko, indeks, grupa, data
- Każde zadanie: treść, kod, testy, opis, przykładowe wejście i wyjście
- Co najmniej jeden test integracyjny na laboratorium
- Opis mówi, co robi program i dlaczego tak

Zadania na lab 1

- Dwie liczby: suma, różnica, iloczyn i iloraz, jeżeli da się go policzyć
- Rok świetlny w czasie i kilometrach, ze zmiennych i stałych
- Sekundy w godzinie, dobie, roku i w Twoim życiu
- Cale na centymetry: 1 cal = 2,54 cm
- Średnia prędkość: 30 km w 15 min, potem 30 km w 12 min

Materiały

- Dokumentacja języka: swift.org/documentation
- The Swift Programming Language: docs.swift.org/swift-book
- A Swift Tour: przegląd języka z przykładami
- Toolchain i SPM: swift.org/getting-started
- Swift Testing: developer.apple.com/documentation/testing

Źródła

- [1] The Swift Programming Language, A Swift Tour, The Swift Project. docs.swift.org/swift-book (dostęp: 19.10.2026)
- [2] Swift Testing, README repozytorium swiftlang/swift-testing. github.com/swiftlang/swift-testing (dostęp: 19.10.2026)
- [3] Swift Testing, dokumentacja Apple Developer. developer.apple.com/documentation/testing (dostęp: 19.10.2026)